



ОНЛАЙН-ОБРАЗОВАНИЕ

Проверить включена ли запись?



Меня хорошо слышно && видно?



Напишите в чат, если есть проблемы!

Ставьте ☐+ если все хорошо

Ставьте ☐- если есть проблемы

JPQL, Spring ORM, DAO на основе Spring ORM + JPA



Дворжецкий Юрий
Петрелевич Сергей



Куценко Виталий
Оруджев Александр

- ❖ API JPA
- ❖ Что делает фреймворк «Spring ORM»
- ❖ DAO на JPA, DAO на Spring ORM + JPA
- ❖ JPQL
- ❖ Тестирование



❖ Наконец-то Spring

❖ Домашка наконец-то есть

Поехали?

01

API JPA, Spring ORM



- ❖ JPA – прослойка между Вашим кодом и ORM фреймворком
- ❖ JPA – это аннотации маппинга
- ❖ JPA – это кое-какие классы (API)

```
@Entity // Указывает, что данный класс является сущностью
@Table(name = "otus_students") // Задаст имя таблицы, на которую будет отображаться сущность
public class OtusStudent {
    @Id // Позволяет указать какое поле является идентификатором
    @GeneratedValue(strategy = GenerationType.IDENTITY) // Стратегия генерации идентификаторов
    private long id;

    // Задаст имя и некоторые свойства поля таблицы, на которое будет отображаться поле сущности
    @Column(name = "name", nullable = false, unique = true)
    private String name;

    // Указывает на связь между таблицами "один к одному"
    @OneToOne(cascade = CascadeType.ALL, orphanRemoval = true)
    // Задаст поле, по которому происходит объединение с таблицей для хранения связанной сущности
    @JoinColumn(name = "avatar_id")
    private Avatar avatar;

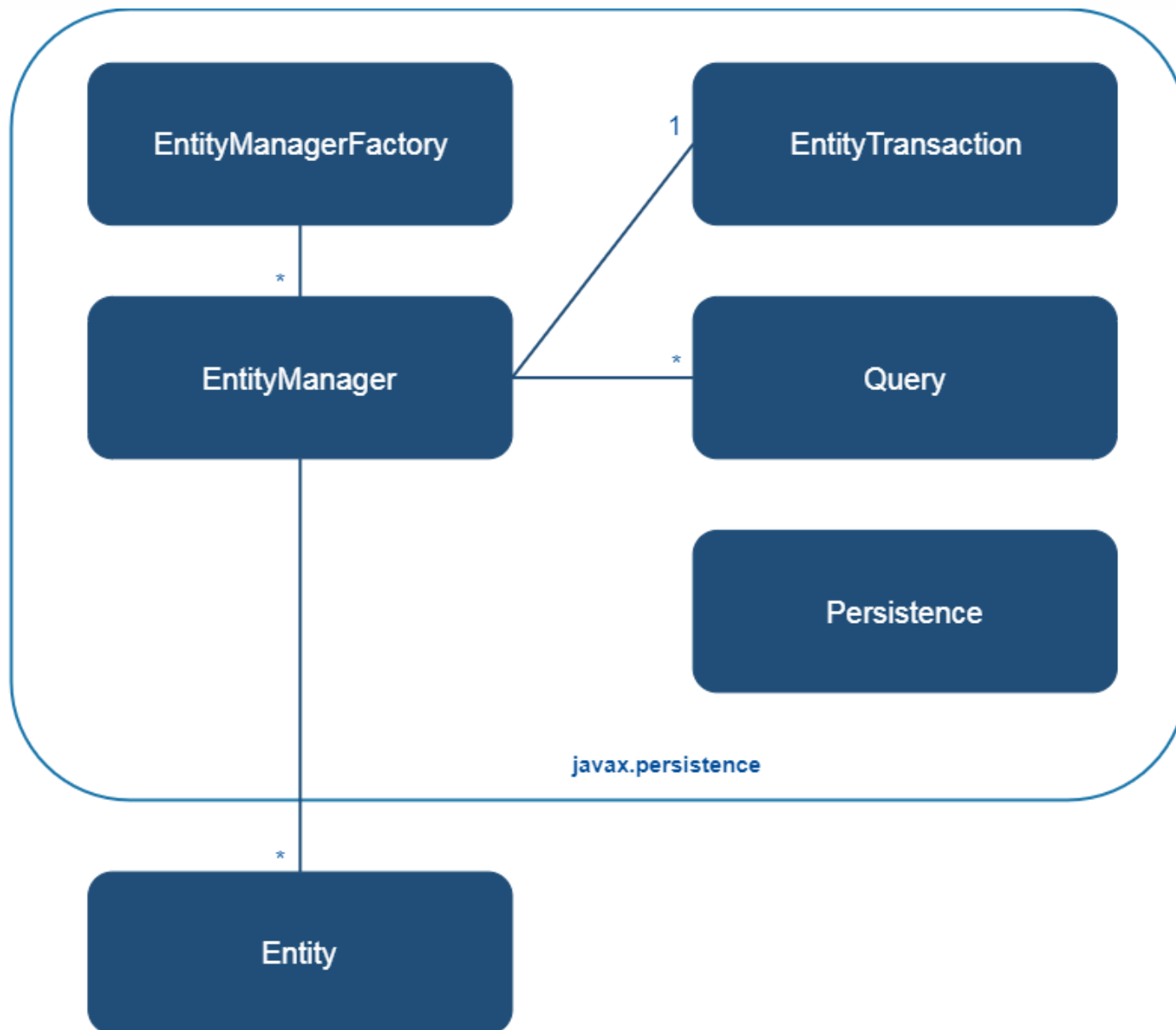
    // Указывает на связь между таблицами "один ко многим"
    @OneToMany(cascade = CascadeType.ALL, fetch = FetchType.LAZY, orphanRemoval = true)
    @JoinColumn(name = "student_id")
    private List<EMail> emails;

    // Указывает на связь между таблицами "многие ко многим"
    @ManyToMany(fetch = FetchType.LAZY, cascade = CascadeType.PERSIST)
    // Задаст таблицу связей между таблицами для хранения родительской и связанной сущностью
    @JoinTable(name = "student_courses", joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id"))
    private List<Course> courses;
}
```

- ❖ JPA – старинный стандарт
- ❖ По стандарту по идее предполагалось наличие META-INF/persistence.xml
- ❖ Он содержал persistence-unit-ы (БД + набор entity)* их может быть много
- ❖ Список entity
- ❖ И настройки JPA Vendor-a

```
<?xml version="1.0" encoding="UTF-8" ?>
<persistence xmlns="http://java.sun.com/xml/ns/persistence"
             xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
             xsi:schemaLocation="http://java.sun.com/xml/ns/persistence
http://java.sun.com/xml/ns/persistence/persistence_1_0.xsd" version="1.0">

  <persistence-unit name="my-pu">
    <description>My Persistence Unit</description>
    <provider>com.objectdb.jpa.Provider</provider>
    <mapping-file>META-INF/mappingFile.xml</mapping-file>
    <jar-file>packedEntity.jar</jar-file>
    <class>sample.MyEntity1</class>
    <class>sample.MyEntity2</class>
    <properties>
      <property name="javax.persistence.jdbc.url" value="objectdb://localhost/my.odb"/>
      <property name="javax.persistence.jdbc.user" value="admin"/>
      <property name="javax.persistence.jdbc.password" value="admin"/>
    </properties>
  </persistence-unit>
</persistence>
```



- ❖ Persistence (соответствует persistence.xml) – главный контекст для получения EntityManagerFactory
- ❖ EntityManagerFactory (соответствует Persistence unit) – создаёт EntityManager + отвечает за метаданные
- ❖ EntityManager (PersistenceContext) – главный класс, с которым происходит работа
- ❖ В один момент времени EntityManager соответствует ровно одна EntityTransaction
- ❖ Из EntityManager создаётся запрос (Query)

@Repository

```
public class OtusStudentRepositoryJpa implements OtusStudentRepository {  
    private EntityManagerFactory emf;
```

```
    @PersistenceUnit(unitName = "my-pu")
```

```
    public void setEmf(EntityManagerFactory emf) {  
        this.emf = emf;  
    }
```

@Override

```
    public OtusStudent save(OtusStudent student) {  
        EntityManager em = emf.createEntityManager();  
        em.getTransaction().begin();  
        OtusStudent s = em.merge(student);  
        em.getTransaction().commit();  
        return s;  
    }
```

```
}
```

```
@Repository
public class OtusStudentRepositoryJpa implements OtusStudentRepository {

    @PersistenceContext
    private final EntityManager em;

    @Override
    public OtusStudent save(OtusStudent student) {
        return em.merge(student);
    }
}
```



```
@Repository
public class OtusStudentRepositoryJpa implements OtusStudentRepository {

    @PersistenceContext
    private final EntityManager em;

    @Transactional // Для примера. Лучше повесить над методом сервиса
    @Override
    public OtusStudent save(OtusStudent student) {
        return em.merge(student);
    }
}
```

Repository

Entity + Mapping

ORM framework (Hibernate)

JDBC

DB

Repository

Entity + JPA

JPA

Hibernate as JPA Vendor

JDBC

DB

Repository

Entity + JPA

JPA

Spring ORM

Hibernate as JPA Vendor

JDBC

DB

Вопросы?

(ставим “+” если есть вопрос и “-”
если нет)

02

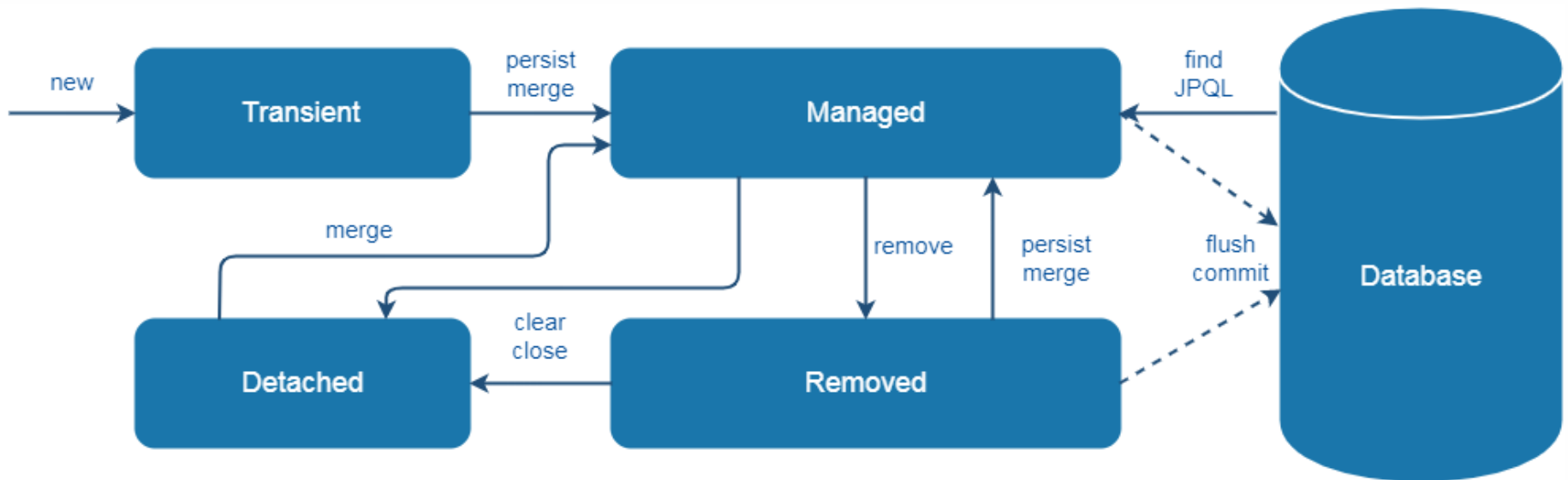
JPA – EntityManager



- ❖ Persistence Context – кэш первого уровня для объектов, которые сохраняются или загружаются в/из БД в рамках сессии/транзакции
- ❖ В применении к данному контексту сущность может находиться в четырех состояниях
 - Transient – новый объект, его нет ни в БД, ни в кэше первого уровня (контексте)
 - Persistent – объект есть в БД и в контексте
 - Detached – объект есть в БД, но отсутствует в контексте
 - Removed – аналогичен Detached, но объекта нет в БД

- Так или иначе, в JPA за все операции над сущностями отвечает EntityManager
- По классике создается, с помощью EntityManagerFactory на основе настроек из "META-INF/persistence.xml" (в ресурсах)
- Но у нас он будет создаваться Spring ORM-ом на основе настроек из application.yml и метаданных сканированных сущностей
- И внедряться через @PersistenceContext в наши репозитории

- **find** – поиск и загрузка объекта по его id. Объект сразу получается в состоянии persistent
- **persist** – меняет состояние объекта из transient в persistent. Выполняет insert. Бросает PersistentObjectException если задан id
- **merge** – меняет состояние объекта из transient **или detached** в persistent. Для transient работает аналогично persist. Для detached выполняет загрузку из БД и обновляет в контексте. По завершении сессии или коммита транзакции выполняет update
- **remove** – удаление объекта из БД и контекста. Меняет состояние на transient. IllegalArgumentException если объект в состоянии detached
- **refresh** – обновление объекта из БД если он был изменен в другом месте уже после загрузки
- **detach** – отключает объект от контекста. Меняет состояние на detached
- **createQuery** – создание объекта запроса
- **getEntityGraph** – получение, ранее определенного графа объектов



- В Hibernate есть специальный механизм, “Dirty Checking”
- Это значит, что для всех сущностей, под управлением контекста хранятся копии всех их свойств
- При флаше (flush) сессии, происходит сравнение итоговых значений свойств сущностей с их исходными копиями и если они не совпадают, то формируется запрос на обновление этих сущностей в БД
- Flush происходит в момент закрытия сессии, коммита транзакции или в момент явного вызова одноименного метода
- А теперь представим, что у нас в контексте лежит миллион сущностей...



- Из-за dirty checking, нужно хранить больше данных в контексте
- Если сущностей много, то и памяти нужно тоже много
- Но самое плохое, что перед флашем нужно проверить все сущности на предмет изменений, что может ударить по производительности
- Поэтому мы можем отключить данный механизм, выставив у сессии FlushMode или просто отсоединив сущности от контекста с помощью detach

Давайте освежим в памяти нашу объектную модель и маппинг



```
@Entity // Указывает, что данный класс является сущностью
@Table(name = "otus_students") // Задаст имя таблицы, на которую будет отображаться сущность
public class OtusStudent {
    @Id // Позволяет указать какое поле является идентификатором
    @GeneratedValue(strategy = GenerationType.IDENTITY) // Стратегия генерации идентификаторов
    private long id;

    // Задаст имя и некоторые свойства поля таблицы, на которое будет отображаться поле сущности
    @Column(name = "name", nullable = false, unique = true)
    private String name;

    // Указывает на связь между таблицами "один к одному"
    @OneToOne(cascade = CascadeType.ALL, orphanRemoval = true)
    // Задаст поле, по которому происходит объединение с таблицей для хранения связанной сущности
    @JoinColumn(name = "avatar_id")
    private Avatar avatar;

    // Указывает на связь между таблицами "один ко многим"
    @OneToMany(cascade = CascadeType.ALL, fetch = FetchType.LAZY, orphanRemoval = true)
    @JoinColumn(name = "student_id")
    private List<EMail> emails;

    // Указывает на связь между таблицами "многие ко многим"
    @ManyToMany(fetch = FetchType.LAZY, cascade = CascadeType.PERSIST)
    // Задаст таблицу связей между таблицами для хранения родительской и связанной сущностью
    @JoinTable(name = "student_courses", joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id"))
    private List<Course> courses;
}
```

```
@Repository
public class OtusStudentRepositoryJpa implements OtusStudentRepository {
    @PersistenceContext
    private EntityManager em;

    @Override
    public OtusStudent save(OtusStudent student) {
        if (student.getId() == 0) {
            em.persist(student);
            return student;
        } else {
            return em.merge(student);
        }
    }

    @Override
    public Optional<OtusStudent> findById(long id) {
        return Optional.ofNullable(em.find(OtusStudent.class, id));
    }
}
```

Добавить в репозиторий методы

- save
- findByld

Должны начать проходить соответствующие тесты

```
git clone git@github.com:OtusTeam/Spring.git
или
git clone https://github.com/OtusTeam/Spring.git

cd ../<имя группы>/spring-11/jpql-class-work
```

Или сразу смотрим решение в проекте jpql-solution-01



Вопросы?

(ставим “+” если есть вопрос и “-”
если нет)

03

JPQL



- ❖ Создаётся в EntityManager
- ❖ Использует язык запросов JPQL (бизнес-объекты)
- ❖ Бывает Query и TypedQuery<> (лучше использовать typed, когда это возможно)
- ❖ Может возвращать одну сущность или список (getSingleResult / getResultList)
- ❖ Или просто выполнить запрос (executeUpdate)

```
select e from Employee e
```

```
select e from Employee e where e.id = :id
```

```
select e from Employee e where e.salary > 100000
```

```
select e.firstName from Employee e
```

```
select e.firstName, e.lastName from Employee e
```

```
select count(e) from Employee e
```

```
select max(e.salary) from Employee e
```

```
select avg(e.salary) from Employee e
```

```
select new r.o.e.j.d.CitySalary(e.address.city, avg(e.salary))  
from Employee e  
group by e.address.city
```

```
select new r.o.e.j.d.CitySalary(e.address.city, avg(e.salary))  
from Employee e  
group by e.address.city  
order by avg(e.salary)
```

```
select new r.o.e.j.d.CitySalary(e.address.city, avg(e.salary))  
from Employee e  
group by e.address.city  
having avg(e.salary) > 100000  
order by avg(e.salary)
```

```
select e
from Employee e join
    e.projects p1 join
    e.projects p2
where p1.name = :p1 and p2.name = :p2

select new r.o.e.j.d.EmployeeProjects(e, count(p))
from Employee e left join e.projects p
group by e
order by count (p) desc
```

```
select e
from Employee e
where e.firstName in (:name1, :name2)
```

```
select e
from Employee e
where e.firstName in :names
```

```
select e
from Employee e
where e.firstName in (select e2.firstName
                      from Employee e2
                      where e2.lastName = :lastName and e2.id <> :id
                      )
```

```
select e
from Employee e
where e.firstName = (select e2.firstName from Employee e2 where e2.id = :id) and e.id <> :id
```

```
select e
from Employee e
where e.salary < (select e2.salary from Employee e2 where e2.id = :id)
```

```
select e
from Employee e
where e.firstName = any (select e2.firstName from Employee e2 where e2.id <> e.id)
```

```
select e
from Employee e
where e.salary <= all (select e2.salary from Employee e2)
```

```
update Employee e  
set e.salary = 60000  
where e.salary = 30000
```

```
update Employee e  
set e.salary = e.salary * 2  
where e.salary = 30000
```

```
delete  
from Employee e  
where e.department is null
```



```
@Override
public List<OtusStudent> findAll() {
    TypedQuery<OtusStudent> query = em.createQuery("select s from OtusStudent s",
        OtusStudent.class);
    return query.getResultList();
}

@Override
public List<OtusStudent> findByName(String name) {
    TypedQuery<OtusStudent> query = em.createQuery("select s " +
        "from OtusStudent s " +
        "where s.name = :name",
        OtusStudent.class);
    query.setParameter("name", name);
    return query.getResultList();
}
```

Добавить в репозиторий методы

- findAll
- findByName

Должны начать проходить соответствующие тесты

```
git clone git@github.com:OtusTeam/Spring.git
или
git clone https://github.com/OtusTeam/Spring.git

cd ./<имя группы>/spring-11/jpql-class-work
```

Или сразу смотрим решение в проекте jpql-solution-02



Демо

(JPQL – загрузка и контекст)

```
@Override
public void updateNameById(long id, String name) {
    Query query = em.createQuery("update OtusStudent s " +
        "set s.name = :name " +
        "where s.id = :id");
    query.setParameter("name", name);
    query.setParameter("id", id);
    query.executeUpdate();
}

@Override
public void deleteById(long id) {
    Query query = em.createQuery("delete from OtusStudent s where s.id = :id");
    query.setParameter("id", id);
    query.executeUpdate();
}
```

Добавить в репозиторий методы

- updateNameById
- deleteById

Должны начать проходить соответствующие тесты

```
git clone git@github.com:OtusTeam/Spring.git
или
git clone https://github.com/OtusTeam/Spring.git

cd ../<имя группы>/spring-11/jpql-class-work
```

Или сразу смотрим решение в проекте jpql-solution-03



Демо

(JPQL – удаление и изменение)



04

Проблема N+1 запросов



На прошлом занятии мы говорили, что решаем с помощью ORM проблему. Что в итоге?

- ❖ Сами по результатам не ходим
- ❖ Никаких гигантских SQL
- ❖ Сущности загружаем только так
- ❖ Причем вместе со связями
- ❖ Победа?

O T U S

[illegible]

- ❖ Допустим есть некая сущность. Например, наш OtusStudent
- ❖ И она содержит в себе поле, которое тоже сущность. Например, Avatar
- ❖ Мы запрашиваем у БД одним (1) запросом всех студентов и нам возвращается список из N строк
- ❖ Далее, мы для каждого студента делаем запрос в БД за его аватаром
- ❖ В итоге 1 запрос за списком студентов и N запросов за связями. Т.е. N+1
- ❖ При данном подходе запрос 1000 студентов выльется в 1001 запрос к БД
- ❖ По умолчанию Hibernate использует именно этот подход для загрузки через JPQL

- ❖ EntityGraph – позволяет указать какие свойства/поля сущности загружать с ней в одном запросе
- ❖ Перебивает LAZY
- ❖ Обычно задается с помощью аннотации @NamedEntityGraph над сущностью
- ❖ Может быть применен к объекту Query с помощью метода setHint



```
@NamedEntityGraph(name = "avatars-entity-graph",
    attributeNodes = {@NamedAttributeNode("avatar")})
public class OtusStudent {
    ...
}

@Override
public List<OtusStudent> findAll() {
    EntityGraph<?> entityGraph = em.getEntityGraph("avatars-entity-graph");

    TypedQuery<OtusStudent> query = em.createQuery("select s from OtusStudent s",
        OtusStudent.class);

    query.setHint("javax.persistence.fetchgraph", entityGraph);

    return query.getResultList();
}
```

- ❖ JOIN FETCH – позволяет загружать дочерние сущности в одном запросе с родительской
- ❖ Это часть JPQL запроса



```
@Override
public List<OtusStudent> findAll() {
    EntityGraph<?> entityGraph = em.getEntityGraph("avatars-entity-graph");

    TypedQuery<OtusStudent> query = em.createQuery("select distinct s from " +
                                                    "OtusStudent s join fetch s.emails ",
                                                    OtusStudent.class);

    query.setHint("javax.persistence.fetchgraph", entityGraph);

    return query.getResultList();
}
```

- ❖ `@Fetch(FetchMode.SUBSELECT)` – указывает фреймворку, что нужно отдельно поднять список дочерних сущностей из БД, чтобы потом переженить со списком родительских
- ❖ Вешается над полем родительской сущности
- ❖ Это аннотация Hibernate



```
@Entity
@Table(name = "otus_students")
public class OtusStudent {
    ...
    // Все данные таблицы "courses" будут загружены в память отдельным запросом и
    // соединены с родительской сущностью
    @Fetch(FetchMode.SUBSELECT)
    @ManyToMany(targetEntity = Course.class, fetch = FetchType.LAZY,
        cascade = CascadeType.PERSIST)
    @JoinTable(name = "student_courses",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id"))
    private List<Course> courses;
}
```


- ❖ `@BatchSize(size = N)` – указывает фреймворку, что нужно делать запрос за связями для каждой N записей набора данных родительских сущностей
- ❖ Работает для FETCH-режима по умолчанию (`@Fetch(FetchMode.SELECT)`)
- ❖ Это тоже аннотация Hibernate



```
@Entity
@Table(name = "otus_students")
public class OtusStudent {
    ...
    @Fetch(FetchMode.SUBSELECT) // Не обязательно. Тут для наглядности
    // Для каждых 5 родительских сущностей будет сделан один запрос за курсами
    @BatchSize(size = 5)
    @ManyToMany(targetEntity = Course.class, fetch = FetchType.LAZY,
        cascade = CascadeType.PERSIST)
    @JoinTable(name = "student_courses",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id"))
    private List<Course> courses;
}
```

Решить проблему N+1

- для поля avatar с помощью @EntityGraph
- для поля emails с помощью JOIN FETCH
- для поля courses с помощью @Fetch(FetchMode.SUBSELECT) или @BatchSize(size = 5)
- Должны начать проходить соответствующие тесты
- EXPECTED_QUERIES_COUNT сделать равным 2 (SUBSELECT) или 3 (BatchSize)

```
git clone git@github.com:OtusTeam/Spring.git
или
git clone https://github.com/OtusTeam/Spring.git

cd ./<имя группы>/spring-11/jpql-class-work
```

Или сразу смотрим решение в проектах

```
jpql-solution-04
jpql-solution-05
jpql-solution-06
jpql-solution-final
```



А что с LazyInitializationException?

```
@Transactional(readOnly = true)
@Override
public List<OtusStudentDto> findAll() {
    List<OtusStudent> students = studentRepository.findAll();
    return students.stream.map(studentConverter::toDto).toList();
}
```

Вопросы?

(ставим “+” если есть вопрос и “-”
если нет)

- Какой тип решения N+1 мы будем использовать для автора и жанра, когда их у книги по одному?

- Какой тип решения N+1 мы будем использовать для автора и жанра, когда их у книги по одному?
- Какой тип решения N+1 мы будем использовать для жанров, когда их у книги много?

Демо

(JPQL – пагинация)

05

Тестирование



Тестируем с `@DataJpaTest`. Он нам создаст:

- Кусок контекста, ответственный за persistence layer
- `TestEntityManager` (можно внедрять и пользоваться)
- Транзакцию в начале каждого теста (и ее rollback в конце)

Еще есть аннотация `@EntityScan`. С помощью нее можно задать пакет для сканирования сущностей. Это нужно редко. Только если мы сильно ограничили контекст

```
@DisplayName("Репозиторий на основе Jpa для работы со студентами ")
@DataJpaTest
@Import(OtusStudentRepositoryJpaImpl.class)
class OtusStudentRepositoryJpaImplTest {

    private static final long FIRST_STUDENT_ID = 1L;

    @Autowired
    private OtusStudentRepositoryJpaImpl repositoryJpa;

    @Autowired
    private TestEntityManager em;

    @DisplayName(" должен загружать информацию о нужном студенте по его id")
    @Test
    void shouldFindExpectedStudentById() {
        val optionalActualStudent = repositoryJpa.findById(FIRST_STUDENT_ID);
        val expectedStudent = em.find(OtusStudent.class, FIRST_STUDENT_ID);
        assertThat(optionalActualStudent.isPresent().get()
            .isEqualToComparingFieldByField(expectedStudent);
    }
}
```

Вопросы?

(ставим “+” если есть вопрос и “-”
если нет)

06

Домашнее задание



Задание:

- ❖ Перевести приложение каталог книг в библиотеке на репозитории, использующие в качестве технологии доступа к данным, Spring ORM, JPA и Hibernate
- ❖ Дополнить доменную модель сущностью комментария к книге
- ❖ Задание выполняется на базе предыдущей работы, но также имеет заготовку содержащую тесты, проверяющие некоторые из требований к заданию
- ❖ Решение должно удовлетворять нижеуказанным требованиям.

Требования к использованию заготовки:

- ❖ Тесты из заготовки должны остаться неизменными (за исключением, возможно, имени пакета)

- ❖ Использовать Spring ORM, JPA и Hibernate (EntityManager) в качестве технологии доступа к данным
- ❖ Spring Data пока использовать нельзя
- ❖ Добавить сущность "комментария к книге" (много комментариев к одной книге)
- ❖ Реализовать CRUD операции для новой сущности. Получение всех комментариев делать не нужно. Только конкретного - комментария по id и всех комментариев по конкретной книге по ее id
- ❖ Загрузка связей сущностей не должна приводить к большому количеству запросов к БД или избыточному по объему набору данных (проблема N+1 и проблема произведения таблиц)
- ❖ DDL через Hibernate должно быть отключено
- ❖ LAZY-связи не должны присутствовать в equals/hashCode/toString. В т.ч. за счет @Data
- ❖ Проблема N+1 должна быть решена теми же способами, что и в прошлой работе, но уже через инструменты, предоставляемыми ORM

- ❖ Аннотация `@Transactional` должна присутствовать только на методах сервиса
- ❖ Покрыть все репозитории тестами, используя H2 базу данных и `@DataJpaTest`
- ❖ В тестах репозиторий, для проверки или подготовки данных использовать `TestEntityManager` (только методы не требующие написания запросов). Сами репозитории для этого использовать нельзя
- ❖ Написать интеграционные тесты сервисов книг и комментариев, где проверить, что доступ к связям, которые используются снаружи сервисов не вызывают `LazyInitializationException`
- ❖ Не забыть учесть в тестах, что тестовый контекст кэшируется, т.е. должны проходить как все тесты разом, так и по отдельности
- ❖ Для приема работы тесты из заготовки должны проходить
- ❖ Без фанатизма)
- ❖ Заготовка для выполнения работы:
<https://github.com/OtusTeam/Spring/tree/master/templates/hw06-jpa>

- ❖ Задание сдаётся в виде ссылки на pull-request в чат с преподавателем в личном кабинете ОТУС, а не в Telegram
- ❖ В pull-request должно присутствовать только то что касается текущей работы. Временные файлы и файлы IDE не должны попадать в PR.
- ❖ Инструкция по работе с pull-request-ами находится в документе "Настройка окружения.pdf" в материалах занятия.
- ❖ Вопросы можно задавать в чате ДЗ, но для оперативности рекомендуем делать это в группе Telegram.
- ❖ Код, написанный в данном ДЗ будет использоваться в дальнейших домашних заданиях всех оставшихся блоков курса

Вопросы?

(ставим “+” если есть вопрос и “-”
если нет)

**Пожалуйста, пройдите
опрос**

Спасибо за внимание!

